

```

#
=====
=====
# AI-Driven Corporate Financial Performance
Analysis
# Beginner-friendly Random Forest model
using accounting ratios
#
=====
=====

import pandas as pd
from sklearn.model_selection import
train_test_split
from sklearn.ensemble import
RandomForestRegressor
from sklearn.metrics import
mean_absolute_error, mean_squared_error,
r2_score
import joblib

# -----
-----
# STEP 1: Load your dataset
# Your Excel/CSV should have one row per
company-year, with
# ratio columns + a "Performance_Score"
column (the target)
# -----
-----

```

```

data = pd.read_csv("company_ratios.csv")
print("Preview of data:")
print(data.head())

# -----
# STEP 2: Separate inputs (X) and
# target/output (y)
# X = all the ratios (what the model learns
# FROM)
# y = the performance score (what the model
# learns to PREDICT)
# -----
X = data.drop(["Company", "Year",
               "Performance_Score"], axis=1)
y = data["Performance_Score"]

# -----
# STEP 3: Split into training data and
# testing data
# 80% used to teach the model, 20% kept
# aside to test it
# -----
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.20, random_state=42
)

# -----
# STEP 4: Create and train the Random

```

```

Forest model
# n_estimators = number of "mini-expert"
decision trees
# -----
-----
model =
RandomForestRegressor(n_estimators=200,
random_state=42)
model.fit(X_train, y_train)

# -----
-----
# STEP 5: Test the model – see how close
its guesses are
# -----
-----
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\n--- Model Performance ---")
print(f"Mean Absolute Error (MAE):
{mae:.2f}")
print(f"Mean Squared Error (MSE):
{mse:.2f}")
print(f"R2 Score (closer to 1 = better):
{r2:.2f}")

# -----
-----
# STEP 6: See which ratios matter most
(great for your poster!)

```

```

# -----
importance =
pd.Series(model.feature_importances_,
index=X.columns)
importance =
importance.sort_values(ascending=False)
print("\n--- Which ratios matter most? ---")
print(importance)

# -----
# STEP 7: Save the trained model so you can
reuse/predict later
# -----
joblib.dump(model, "model.pkl")
print("\nModel saved as model.pkl")

# -----
# STEP 8: Predict performance for a NEW
company (example)
# Replace values below with a real
company's ratios
# -----
new_company = pd.DataFrame([{
    "Current_Ratio": 1.45,
    "Debt_Equity_Ratio": 0.35,
    "Net_Profit_Ratio": 12.36,
    "ROCE": 18.45,
    "Gross_Profit_Ratio": 28.66,

```

```
"Total_Asset_Turnover": 0.96,  
"Inventory_Turnover": 5.32,  
"Quick_Ratio": 1.12,  
"ROA": 9.21,  
"ROE": 16.32,  
"Interest_Coverage_Ratio": 3.45,  
"Operating_Profit_Margin": 19.25  
})  
  
prediction = model.predict(new_company)  
print(f"\nPredicted performance score for  
new company: {prediction[0]:.2f}")
```